

The Million Song Dataset (MSD)

Thomas Waldin

TABLE OF CONTENTS

1. BACKGROUND.....	1
2. PROCESSING	1
3. AUDIO SIMILARITY	4
4. SONG RECOMMENDATIONS	16
5. CONCLUSIONS	21
6. REFERENCES.....	21

1. BACKGROUND

This report presents a comprehensive analysis of the Million Song Dataset, using Apache Spark to implement machine learning algorithms for music classification and recommendation. The Million Song Dataset is one of the largest publicly available collections of audio features for one million songs, released between 1922 and 2011.

This analysis focuses on two main tasks: classification of song genre with audio features, and recommendations based on taste profiles of users.

The significant challenges of these tasks were managing unbalanced classes and accurately assessing generated recommendations.

2. PROCESSING

The datasets were first explored at a high level using HDFS commands. A directory tree showing the organisation of the datasets is presented in Figure 1.

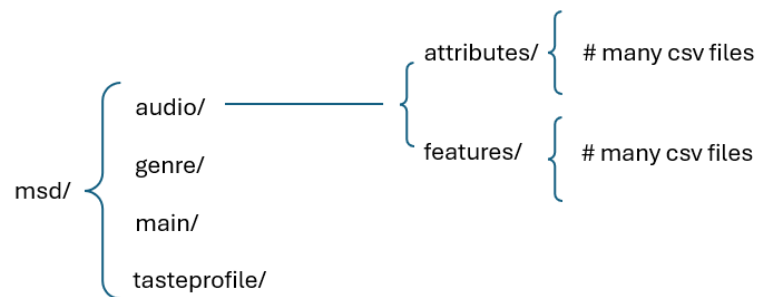


Figure 1. A directory tree showing how the datasets are organized.

Table 1 gives an overview of the structure of the datasets, including sizes and formats.

Table 1. *The names, sizes, formats, data types, and number of rows in each of the datasets.*

<u>Dataset Name</u>	<u>Format</u>	<u>Size (MB)</u>	<u>Rows</u>	<u>Cols</u>	<u>Data Types</u>
main_analysis	csv	55.9	1000000	31	int, double, string
main_metadata	csv	118.4	1000000	20	int, double, string
audio_attr_jmir-spectral-v1	csv	<1	16	2	string
audio_attr_jmir-area-of-moments-v1	csv	<1	20	2	string
audio_attr_jmir-lpc-v1	csv	<1	20	2	string
audio_attr_jmir-methods-of-moments-v1	csv	<1	10	2	string
audio_attr_jmir-mfcc-v1	csv	<1	26	2	string
audio_attr_jmir-spectral-derivatives-v1	csv	<1	16	2	string
audio_attr_marsyas-timbral-v1	csv	<1	124	2	string
audio_attr_mvd-v1	csv	<1	420	2	string
audio_attr_rh-v1	csv	<1	60	2	string
audio_attr_rp-v1	csv	<1	1440	2	string
audio_attr_ssd-v1	csv	<1	168	2	string
audio_attr_trh-v1	csv	<1	420	2	string
audio_attr_tssd-v1	csv	<1	1176	2	string
audio_jmir_spectral	csv	51.5	994615	17	double, string
audio_jmir_area_moments	csv	66.1	994615	21	double, string
audio_jmir_lpc	csv	53.3	994615	21	double, string

audio_jmir_methods_moments	csv	36.1	994615	11	double, string
audio_jmir_mfcc	csv	71.3	994615	27	double, string
audio_jmir_spectral_derivatives	csv	51.5	994615	17	double, string
audio_marsyas_timbral	csv	417.5	994993	125	double, string
audio_mvd	csv	1342.5	994180	422	double, string
audio_rh	csv	242.8	994180	62	double, string
audio_rp	csv	4184.3	994180	1441	double, string
audio_ssd	csv	647.7	994180	170	double, string
audio_trh	csv	1501	994180	422	double, string
audio_tssd	csv	4031	994180	1177	double, string
tasteprofile_matches	txt	<1	938	1	string
tasteprofile_mismatches	txt	2	19094	1	string
tasteprofile_triplets	tsv	493.2	48373578	3	int, string
genre_topMAGD	tsv	10.6	406426	2	string
genre_MASD	tsv	8.4	273935	2	string
genre_MAGD	tsv	11.1	422713	2	string

It is interesting to note that the audio features datasets all contain 994180 rows, suggesting ~6000 songs are missing from these datasets.

The audio features and audio feature attributes datasets were loaded and examined. It appears that each attributes table has a name and type that corresponds to each column in the associated features table. These are stored separately, as the attributes act as metadata for the features. The attribute datasets are small and can be loaded and examined quickly, allowing information about the features to be accessed without loading the full large feature datasets.

As the features and feature attributes datasets are named consistently and share the same prefix, a StructType can be created automatically to define a schema for the feature datasets.

First, a dictionary was created, mapping types in the feature attributes table to `pyspark.sql.types` objects. A function was then defined that took the attribute dataset path as an input, loaded the dataset, and iterated over it to gain column names and types, which were mapped to `pyspark.sql.types` objects using the dictionary. The function returned a `StructType` to be used as a schema when loading the associated feature dataset.

The jMIR methods of moments (all) dataset was loaded to test the function, which worked appropriately, renaming columns with feature names and casting the observations as the appropriate types. While these column names are descriptive and accurate, their length makes them inconvenient to use as column names and redundant as the information is available in the attributes table if required. To rename the columns, another function was defined. Each column in the dataset was renamed, including a prefix denoting the set of features (methods of moments becomes 'mom') and then simply the number of the column, which was achieved by passing in the prefix as an argument and appending the new name to a list once constructed. This makes it clear which feature set the column belongs to if all the sets were merged, and the specific feature can be identified if required by referring to the attribute table. The column denoting the track ID was excluded from this process, such that it may be used when merging the datasets.

3. AUDIO SIMILARITY

There are many audio feature datasets within the million-song dataset that have varying levels of detail. The scope of the assignment is limited to the following audio feature datasets:

- msd-jmir-area-of-moments-all-v1.0
- msd-jmir-lpc-all-v1.0
- msd-jmir-spectral-all-all-v1.0
- msd-marsyas-timbral-v1.0

jMIR is an open-source software for music classification. The area of moments features captures the distribution characteristics of spectral energy. Linear predictive coding (LPC) features model the spectral envelope of audio signals. Spectral features cover various frequency domain characteristics.

Marsyas is another audio software. Timbral features specifically target perceptual audio qualities.

These datasets were loaded using the prior defined functions to generate schema and rename columns sensibly. These datasets were merged on the track ID column via three left joins. The

jMIR datasets required the stripping of quotation marks on this column so that the track IDs lined up with the IDs in the Marsyas dataset.

Descriptive statistics were generated for each audio column across the dataset. While there were too many columns to in the dataset for these to manually inspected, it was clear that some features were not distributed in the same way. Descriptive statistics for the first four columns are presented in Table 2.

Table 2. *Descriptive statistics for a small set of features, demonstrating the differences in distribution.*

<u>Index</u>	<u>Count</u>	<u>Mean</u>	<u>Std Dev</u>	<u>Min</u>	<u>Max</u>
aom_1	994579	1.22	0.52	0.0	9.346
aom_2	994579	5500.57	2366.04	0.0	46860.0
aom_3	994579	33817.96	18228.76	0.0	699400.0
aom_4	994579	1.27E8	2.39E8	0.0	7.864E9

Features with different distributions need to be scaled. This will be handled by Spark's built in scalers.

A plot of the correlation between features is presented in Figure 2.

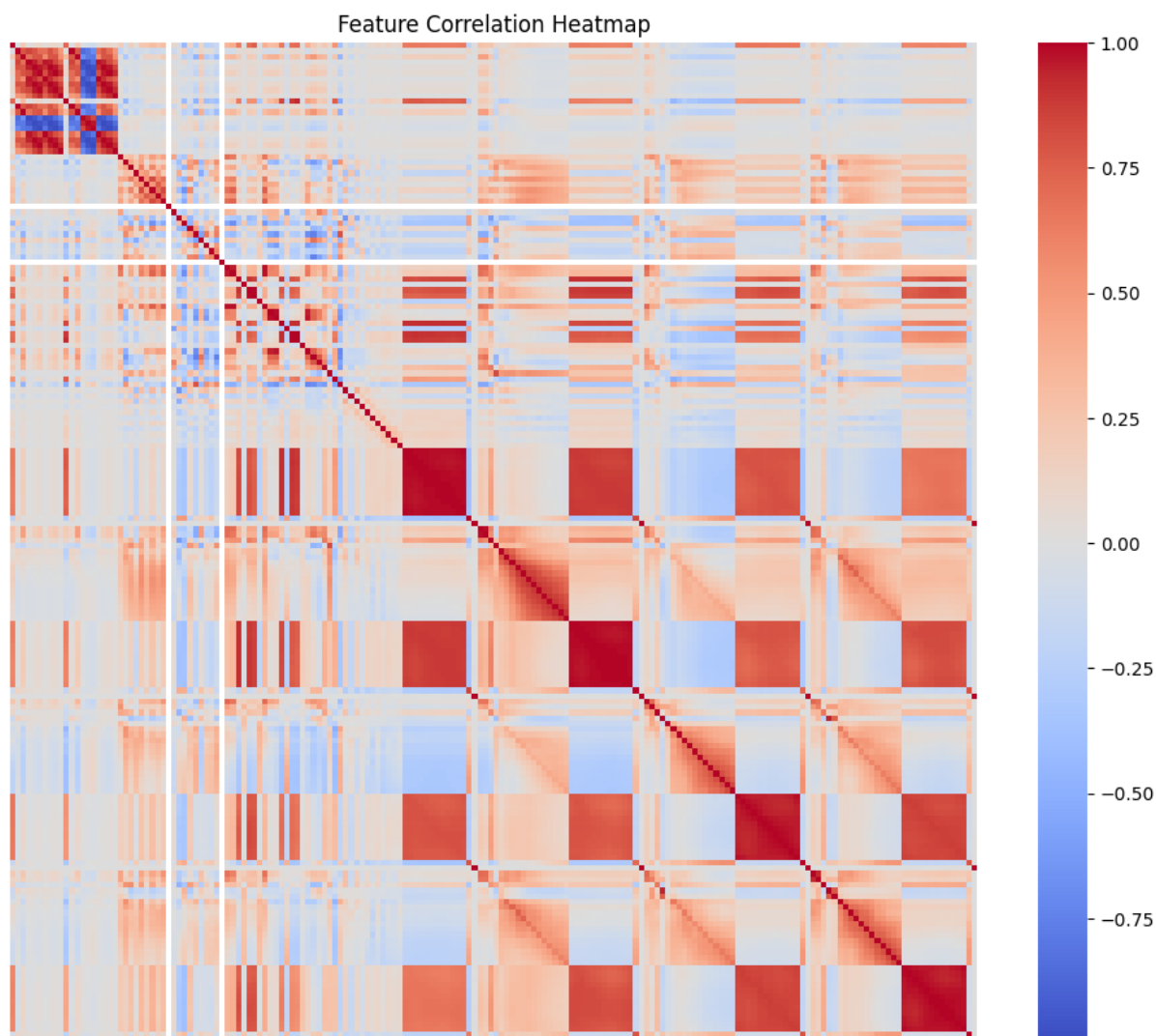


Figure 2. *A heatmap demonstrating correlations between variables. Feature labels have been dropped for clarity, but the plot demonstrates many variables are highly correlated.*

Figure 2 shows that many features are strongly correlated. This is an issue as some classifiers assume independence of features and perform poorly with highly correlated features. It was decided that features with a correlation of 0.9 or higher would be dropped from the dataset.

The MSD all music genre dataset was loaded, and the distribution of genres was visualised in Figure 3.

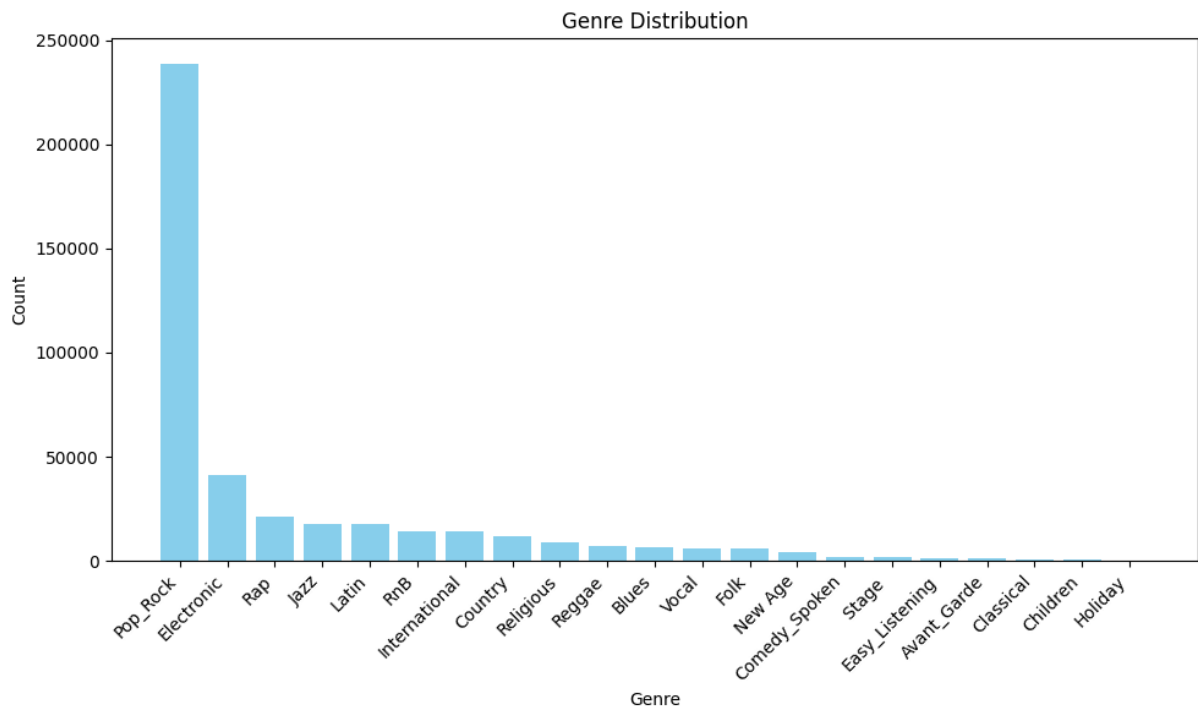


Figure 3. *A visualisation of the distribution of genres in the dataset. From this visualisation it is clear how unbalanced the dataset is.*

Many algorithms are designed to optimise overall accuracy, which leads to favouring majority classes. This can lead to poor recall for underrepresented groups. This will likely be an issue for models trained on this dataset.

The audio features dataset was merged with the genres dataset such that every track was labelled with a genre.

Binary classification models were trained using the following three classification algorithms:

- LogisticRegression
- RandomForestClassifier
- GBTCClassifier

These three classification algorithms are sensible options due to their complementary strengths, which are demonstrated in Table 3.

Table 3. *Strengths and weaknesses of the selected classifiers.*

<u>Algorithm</u>	LogisticRegression	RandomForest	GBTClassifier
<u>Explainability</u>	high	moderate	low
<u>Interpretability</u>	high	moderate	low
<u>Predictive Accuracy</u>	moderate	high	very high
<u>Training Speed</u>	fast	medium	slow
<u>Hyperparameter Tuning</u>	low	medium	high
<u>Handles Dimensionality</u>	no (prone to overfit)	yes	yes
<u>Scaling</u>	yes (sensitive to scale)	no	no

Based on the descriptive statistics and the information in Table 3, the data used to train the LogisticRegression classifier was scaled using Spark's StandardScaler.

The genre column was converted to a new binary column 'isElectronic' that denotes if each track has the genre Electronic or not. The class balance is presented below in Table 4.

Table 4. *The class balance of the binary label 'isElectronic'.*

isElectronic	Count
0 (False)	379932
1 (True)	40660

As can be seen in the above table, the classes are unbalanced, with the positive class only making up around 10% of the data. As mentioned prior, unbalanced classes can lead to poor recall on minority classes.

The data was vectorised then split into training and test sets using an 80 / 20 split. Simple stratified sampling was used to ensure class balance was preserved. The `stat.sampleBy` method was used to sample 80% of each class into a training set. This training set was subtracted from the full dataset to give the remaining 20% of each class in the test set. Simple stratified sampling yields more reliable insights for minority groups as it guarantees that training and test sets maintain the same class balance as the original dataset.

The training set was then downsampled at random such that the positive class became approximately one quarter of the training set. This was to improve the recall of the minority class, by reducing the class imbalance.

Each of the three classification models were trained and evaluated using the test set to produce the following metrics:

- Accuracy
- Precision
- Recall
- AUROC

Accuracy is fairly meaningless in cases with unbalanced classes; however, it is reported for completeness. Precision and recall each offer insight into how classes are misclassified, and AUROC offers a threshold-independent measure of how well a classifier separates classes.

The computed metrics for each classifier are presented in Table 5.

Table 5. *Performance metrics for each of the classification algorithms highlighting their relative performance.*

<u>Classifier</u>	<u>Accuracy</u>	<u>Precision</u>	<u>Recall</u>	<u>AUROC</u>
LogisticRegression	0.907	0.515	0.550	0.870
RandomForest	0.917	0.631	0.334	0.843
GBTClassifier	0.903	0.501	0.538	0.872

The GBTClassifier appeared to be performing the best, given the highest AUROC of 0.872. The LogisticRegression performed similarly with an AUROC of 0.870. Both of these classifiers had similar rates of recall and precision, with recall being slightly better for both, and the LogisticRegression having both higher. The RandomForest classifier had the worst AUROC of

0.843 but the highest precision, and poor recall. In situations where precision is important (avoid false positives) the RandomForest would be the best choice. Where both precision and recall are important (positive class focussed) LogisticRegression may be the best choice. Where overall classification ability is desired, GBTClassifier performs best. In this situation, electronic music was being identified. As no wider context for this classification is provided, overall classification ability is sensible to prioritise, so AUROC is the best metric to compare performance. This suggests that the GBTClassifier is the best performing, but the high AUROC scores suggest all models are good classifiers. It was noted that due to the class imbalance in the data, accuracy is a poor indicator of performance.

The LogisticRegression classifier was used to extend the classification task to predict across all genres. This classifier supports multiple class 'out of the box' with no extra classes or significant configuration required.

Many classifiers do not natively support multiclass classification but can still be used to predict multiple classes, via one-vs-one or one-vs-rest approaches. One-vs-one trains a binary classifier for every pair of classes and predictions are decided by voting. This approach is robust but scales poorly due to the number of required classifiers exploding. One-vs-rest trains one classifier per class and treats all the other classes together as the negative class. At prediction, the class with the strongest positive signal is selected. This approach is simpler and quicker but less robust.

The StringIndexer function was used to encode genres consistently with integer labels. The class balance changed significantly going from binary to multiclass classification. Instead of an approximate 90/10 balance, the classes were incredibly varied. For example, the smallest class was approximately 0.2% of the largest classes, and some classes were effectively balanced.

Simple stratified sampling was used as before to ensure class balance preservation when splitting into training and test sets. As before, simple stratified sampling produces better performance on minority classes by maintaining class balance. The training was repeated using upsampling for classes with less than 1000 observations and downsampling for classes with more than 100000 observations. Due to the large imbalances at both ends of the spectrum, it was decided this would be more effective than just up or downsampling alone. This should improve recall on minority classes. Once again, the data was scaled to account for the various distributions of features.

The LogisticRegression algorithm was trained and assessed by computing the following performance metrics.

- Precision (each class and averages)
- Recall (each class and averages)
- AUROC (each class and weighted average)

Performance metrics can be adapted for multiclass classification simply by applying them to each class and taking averages of metrics. This provides information at the class level, but also metrics to assess the full classifier.

The metrics for the multiclass classification are presented in Table 6.

Table 6. *Performance metrics for each class highlighting their relative performance.*

<u>Class</u>	<u>Precision</u>	<u>Recall</u>	<u>AUROC</u>	<u>Support</u>
Pop Rock	0.786	0.777	0.834	47727
Electronic	0.468	0.617	0.868	8136
Rap	0.568	0.700	0.933	4120
Jazz	0.448	0.589	0.911	3687
Latin	0.325	0.337	0.814	3527
RnB	0.306	0.306	0.851	2910
International	0.201	0.109	0.746	2854
Country	0.274	0.330	0.875	2246
Religious	0.216	0.049	0.793	1766
Reggae	0.500	0.448	0.948	1431
Blues	0.290	0.234	0.893	1318
Vocal	0.291	0.271	0.907	1200
Folk	0.171	0.094	0.854	1078
New Age	0.337	0.307	0.921	808
Comedy Spoken	0.582	0.605	0.949	392

Stage	0.301	0.196	0.913	301
Easy Listening	0.404	0.065	0.881	321
Avant Garde	0.061	0.014	0.854	213
Classical	0.237	0.211	0.901	109
Children	0.055	0.035	0.897	87
Holiday	0.024	0.022	0.867	46
Weighted Average	0.609	0.620	0.849	84277

The weighted average AUROC score was 0.849 which is slightly worse than the binary LogisticRegression (0.870) but still suggests good performance. Within the classes, some have poorer AUROC scores (International 0.75), and some suggest a strong performance (Comedy Spoken 0.95). The recall and precision of the minority classes are very poor, likely due to the class imbalance, but the AUROC scores do not seem to be correlated with class sizes. As with before, there is no wider context suggesting precision or recall should be the most relevant metrics, so AUROC being a measure of the model's classification ability should be assessed. The average AUROC score of 0.849 suggests that the model was performing well. However, the underperforming classes should be considered in any real application of this model.

The AUROC is the most relevant performance metric as stated previously. The AUROC scores for each class are plotted in Figure 4.

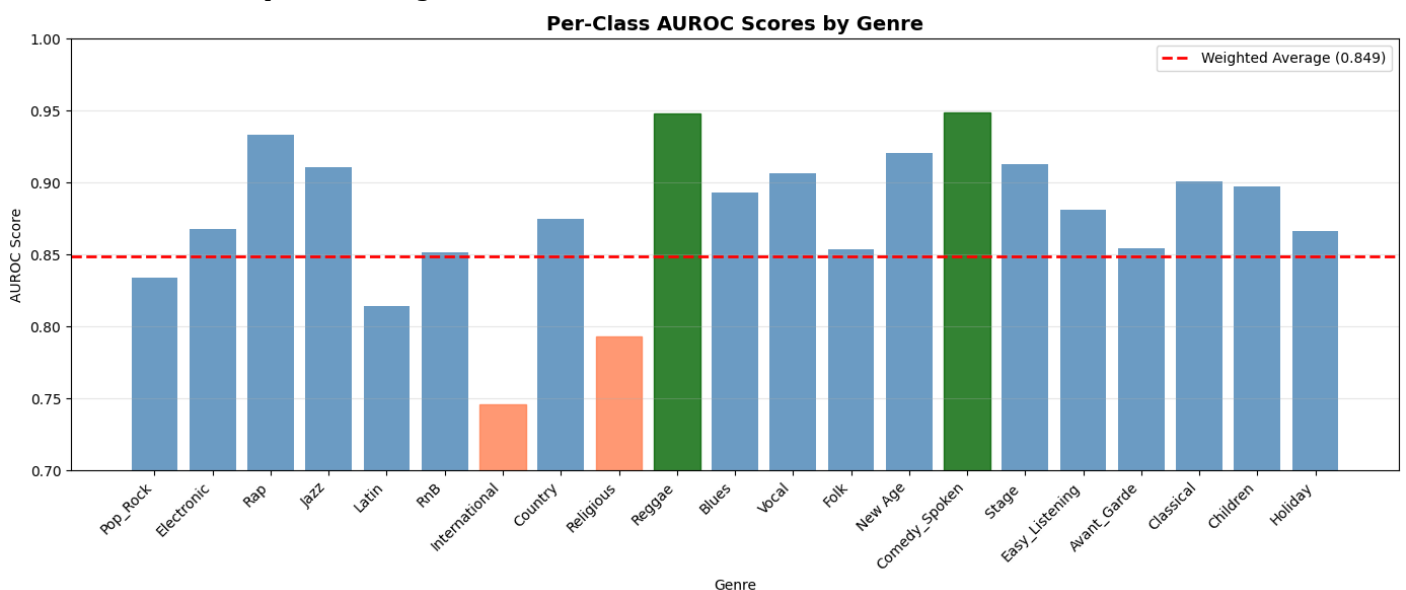


Figure 4. *A visualisation highlighting the most relevant metrics.*

As can be seen in Figure 4, the classifier performed significantly worse on music with the International genre, and performed well on Rap, Reggae, and Spoken Comedy.

In this example, hyperparameter tuning has not been considered, but in a real-life example, this is a key step in training a model. Each of the three models used has different hyperparameters that each affect the model in a different way. While there are many for each model, the most influential ones are presented in Table 7.

Table 7. *A description of hyperparameters for each model.*

<u>Hyperparameter</u>	<u>Description</u>	<u>Effect on Performance</u>
LogisticRegression		
regParam	Regularisation strength (uses L2 by default)	Higher values increase regularisation but risk underfitting.
elasticNetParam	Mix between L1 and L2 regularisation	Helps in feature selection or smoothing.
maxIter	Number of boosting iterations (trees)	More iterations result in a stringer model by risks overfitting.
tol	Convergence tolerance	Controls when to stop iterating.
fitIntercept	Whether to fit an intercept	Usually, true.
standardization	Whether to standardise features	Helps with convergence and scaling issues during optimisations.
threshold	Decision threshold	Can shift recall and precision which is useful in imbalanced classifications.

weightCol	Weights for training instances	Helps balance class distribution. Important for imbalanced class problems.
RandomForestClassifier		
numTrees	Number of trees in the 'forest'	More trees result in better accuracy but slower training.
maxDepth	Depth of each individual tree	Controls complexity of each learner, high complexity risks overfitting.
maxBins	Number of bins to split continuous features	Should be high enough for categorical splits or will hurt performance.
minInstancesPerNode	Minimum data points per node	Prevents too specific splits. Higher values result in more general rules.
featureSubsetStrategy	How many features to consider at each split.	Controls randomness and tree diversity, which can reduce overfitting.
impurity	Criterion for splitting nodes.	Minimal effect of performance.
subsamplingRate	Sample fraction per iteration	Reduces overfitting by introducing randomness.
weightCol	Weights for training instances	Helps balance class distribution. Important for imbalanced class problems.
GBTClassifier		

maxIter	Number of boosting iterations (trees)	More iterations result in a stringer model by risks overfitting.
maxDepth	Depth of each individual tree	Controls complexity of each learner, high complexity risks overfitting.
maxBins	Number of bins to split continuous features	Should be high enough for categorical splits or will hurt performance.
stepSize	Learning rate	Lower results in better generalisation but requires more trees.
minInstancesPerNode	Minimum data points per node	Prevents too specific splits. Higher values result in more general rules.
subsamplingRate	Sample fraction per iteration	Reduces overfitting by introducing randomness.
weightCol	Weights for training instances	Helps balance class distribution. Important for imbalanced class problems.

For the single class classification, many hyperparameters would have been chosen differently.

For the LogisticRegression, tuning the regParam between 0.01 and 0.1 would help balance bias and variance and selecting 0.5 for the elasticNetParam mixes L1 and L2 regularisation which helps with feature sparsity. Using class weights for the weightCol hyperparameter would boost minority class recall and tuning the threshold would adjust precision and recall trade-off as required.

Increasing the numTrees to 100-300 in the RandomForest classifier improves the stability and reduces variance. Limiting the max depth to 10-20 would control the model complexity and prevent overfitting. weightCol should also use class weights.

The GBTClassifier would benefit from an increased maxIter to 50-150, to capture more patterns. Reducing the stepSize to 0.05-0.2 would slow the learning and result in better generalisation. Using 5-10 for maxDepth generalises better.

For the multiclass LogisticRegression, the same hyperparameter selection would have been made as the binary classifier, except without adjusting the threshold, as that cannot be done in a one vs. all situations.

To tune these hyperparameters, cross-validation may be used. This technique involves splitting the testing set into subsets or “folds”. In K-fold cross-validation the dataset is split into K different parts and the model is trained on K-1 folds. The held-out fold is used for validation and this process is repeated K times, with the performance metrics averaged across all folds. This reduces the risk of overfitting to a single train-test split and allows tuning of hyperparameters without ‘data leakage’. A separate test set is still held out for model evaluation and the hyperparameter tuning does not require evaluation of the test set, meaning it gives a better indication of the model’s true ability to generalise to new data.

When tuning hyperparameters, various options can be tested, and the resulting validation metrics can be used to assess the influence. A common approach is a grid search, where a grid of hyperparameters is defined (e.g. for a Random Forest, this could include numTrees = [50, 100, 200], maxDepth = [10, 20, 30], etc.) and cross-validation could be run on all combinations.

It is expected that hyperparameter tuning could improve the performance metrics significantly, 5-10% or more. A model could be performing poorly due to overfitting or underfitting and tuning will reduce this possibility.

4. SONG RECOMMENDATIONS

The taste profile dataset contains information on the songs played by users. This was loaded and examined. It has close to 50 million rows and is half a gigabyte in size. This fits on a single spark partition by default. Repartitioning was useful, as it was known that downstream there are many joins and aggregations required which benefit from more partitions for efficient processing. Caching this dataset was unnecessary as reading it from disk was quick and caches require memory. However, a cache may be useful after transformations, especially on reused datasets.

Descriptive statistics about the dataset were computed and are presented in Table 8.

Table 8. *Descriptive statistics of the taste profile dataset.*

Unique songs	384546
Unique users	1019318
Unique songs played by top user	202
% of songs played by top user	0.05%

Interestingly the top user (by music volume) does not have a high musical diversity. As can be seen in Figure 6, there are many users that have more unique song listens than the top user by volume.

The distribution of song popularity (defined by the number of users plays per song) and distribution of user activity (defined as the number of unique songs played by each user) were each computed. These are visualised in Figures 5-6 respectively.

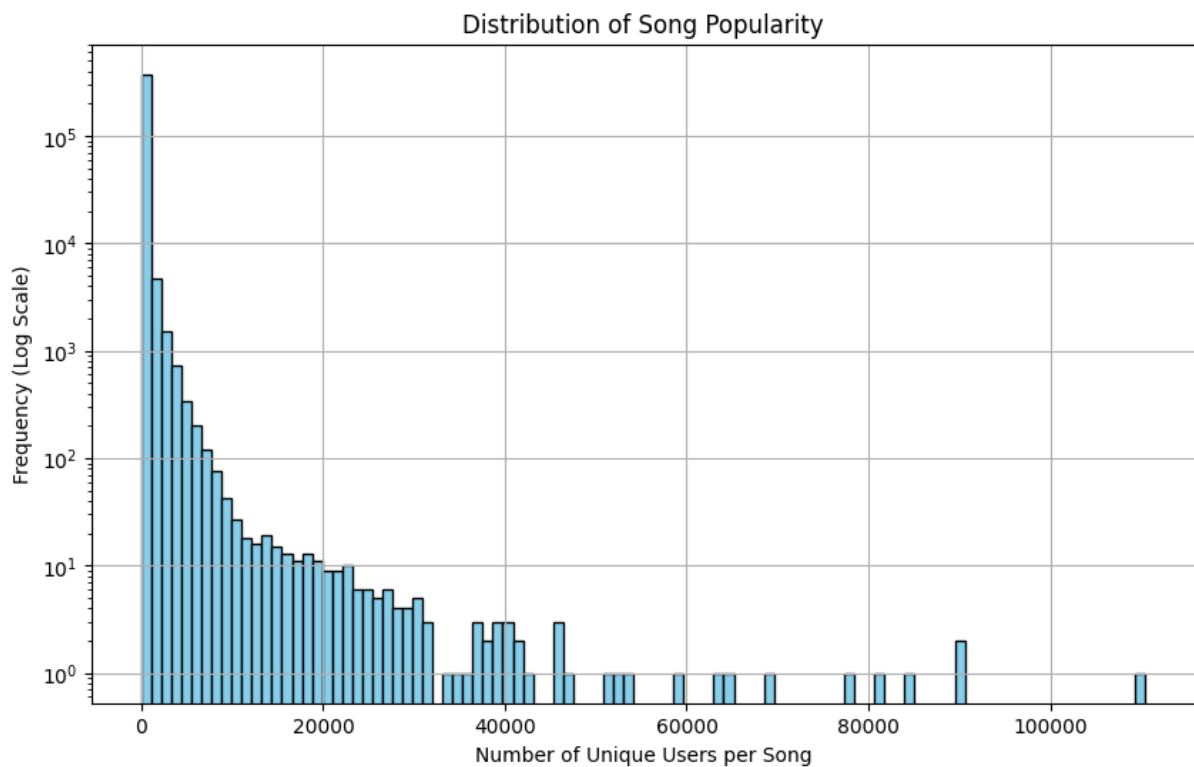


Figure 5. *A visualisation of the distribution of song popularity.*

The distribution of song popularity was strongly skewed to the right, with most songs having very few unique users that have listened to them. This skew is stronger than it appears on the plot due to the logarithmic scale on the y-axis. There are many outliers of this trend, which represent popular songs, with many unique listeners.

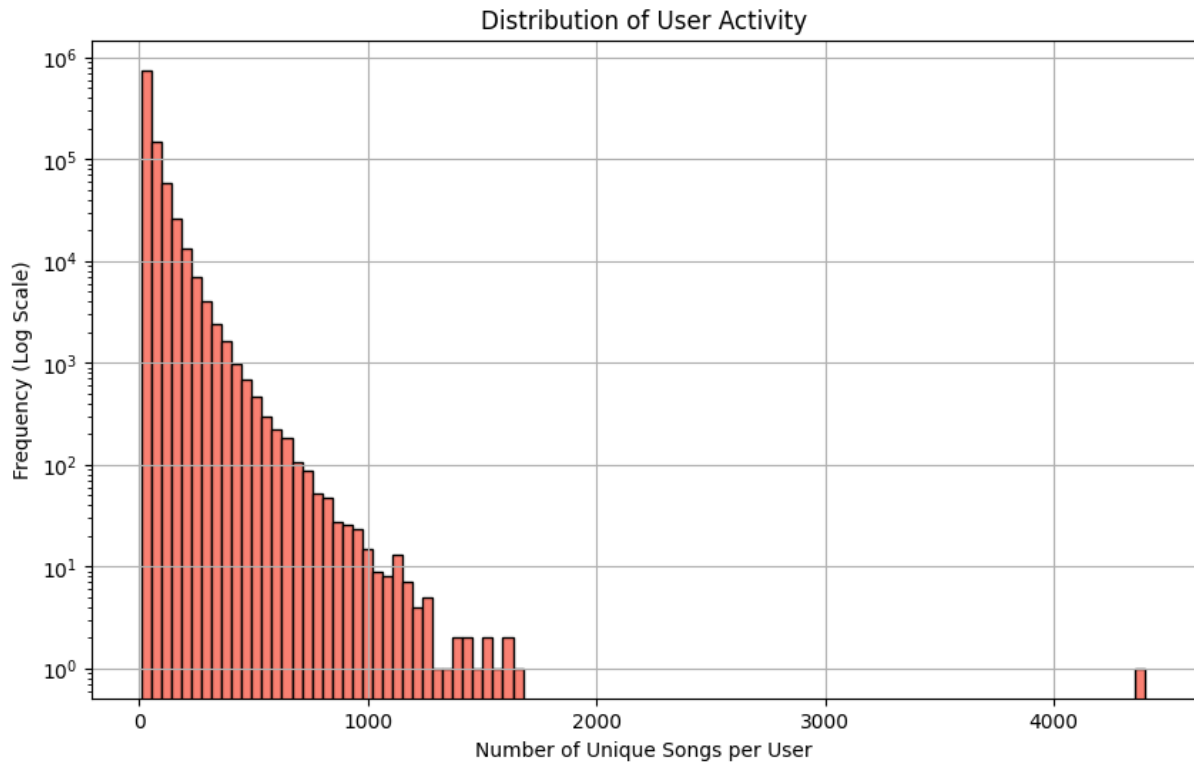


Figure 6. *A visualisation of the distribution of user activity.*

Much like the distribution of song popularity, the distribution of user activity was also strongly skewed to the right, and more so than it appears on the plot due to the logarithmic scale. There are fewer outliers in this distribution, which makes sense, as there are likely more ‘popular songs’ than there are users who listen to a very wide range of music.

A collaborative filtering model determines similarities between users based on their historical data, so in this case songs that have only been played a few times, and users that have only listened to a few songs will likely not provide information that will significantly improve the model. Therefore, songs played fewer than 20 times and users who played fewer than 20 songs were removed from the dataset.

Threshold values of 20 make sense in both cases. 20 interactions provide enough data points to establish meaningful patterns, and removing those below helps avoid the ‘cold start’ problem where the model doesn’t have enough historical data to make meaningful recommendations. This threshold also reduces the data by a significant amount, improving computational efficiency.

The numbers of users and songs excluded with these thresholds are presented in Table 9.

Table 9. *Songs and users removed/remaining with a threshold of 20.*

	Excluded	Remaining
Songs	223373	161173
Users	371510	647808

To obtain these values, the dataframe was grouped by track ID then aggregated by the distinct counts of user ID. This was filtered to only include user counts greater than 20, and then track IDs were selected. This process was repeated to obtain the filtered set of users as well. There is a possibility that in filtering the dataset for users, new tracks may drop below the threshold. This was not a concern, however, as the threshold sensibly reduces the size of the data, so the odd song or user that was incorrectly retained or dropped will not have a great effect.

StringIndexer was used to convert user and song IDs into integer identifiers in a consistent way.

Collaborative filtering suffers from the ‘cold start’ problem. This refers to the challenge of making accurate recommendations for users with no prior interaction data. In this case, it must be ensured that every user in the test set has some user-song plays in the training set as well. Train test split (80/20) was made as normal, but users in test and not in train were identified using a left anti-join of test users on train users. All rows from test that contained users with this issue were removed from the test set and added to the train set.

An implicit matrix factorisation model was trained using alternating least squares (ALS).

A few users were selected by hand from the test set and analysed qualitatively. This is generally a good idea, as the metrics presented in Table 10 only reward direct matches, but this method allows for examination of artists and genres.

One user (ID: 0047af22adb0b100892d343da74460693a448c98) had heavily electronic/dance focused recommendations. The observations fit this trend with mostly electronic artists, with a few hardcore/metal artists. There were matching artists in the observations and recommendations including Simian Mobile Disco, Boys Noize, and Justice. This suggests that the recommendations may be more relevant than the metrics show.

The following metrics were used to assess the recommendations from the collaborative filtering model for $K = 10$:

- Precision @ K
- NDCG
- Mean Average Precision (MAP)

These metrics for the model's performance on the test set are presented in Table 10.

Table 10. *Average metrics for the collaborative filtering model over all users.*

Precision @10	0.029
NDCG @10	0.036
MAP @10	0.014

Precision @K is the fraction of relevant items in all the recommended items. This result states that on average 2.9% of the top ten recommendations are relevant. This metric is limited because it only considers if an item is relevant, ignoring degrees of relevance or position.

Normalised discounted cumulative gain (NDCG) is a measure of how good a ranked list is, by penalising recommendations based on their ranking. A limitation is that due to a log penalty it is difficult to interpret (i.e. what does a score of 0.036 mean?)

Mean average performance (MAP) averages the precision @K at each position, effectively considering the ranking and the precision of the relevant items. A limitation of this metric and all previously discussed metrics is that K is arbitrary. In a real recommendation system however, it may be known how many recommendations can be shown to a user, giving an indication of a sensible K value.

These metrics are all quite poor, but as stated this may not necessarily mean the recommendations are not relevant.

While these metrics are functional for offline systems, there are better options for a system deployed in the real world. Dividing users into groups for A/B testing means that the current or baseline recommendation system can be compared with a new one being deployed. Business oriented online metrics can be used to assess performance of each such as click through rates, conversion rates, return rates, etc. Both implicit (duration, clicks etc.) and explicit feedback such as ratings can be used to assess which recommendation system is better suited for the situation.

5. CONCLUSIONS

This report successfully demonstrated the application of Apache Spark's distributed computing ability to perform large scale machine learning tasks on the Million Song Dataset. A binary classification achieved a good performance (AUROC = 0.872) when classifying the electronic genre and maintained that performance on multiclass classifications (AUROC = 0.849), even when dealing with extremely unbalanced classes. The generated recommendations had more performance metrics, but qualitative analysis suggested the recommendations were still relevant. These tasks provided a significant challenge but also a great opportunity to learn, especially dealing with unbalanced classes and assessing recommendation systems.

6. REFERENCES

Bertin-Mahieux, T., Ellis, D. P. W., Whitman, B., & Lamere, P. (2011). The Million Song Dataset. Columbia University. <https://millionsongdataset.com/>